Big Data Engineering

Complete Study Guide — Phases 6 to 15

Hadoop · HDFS · MapReduce · Hive · HBase

Spark · Kafka · Airflow · MLlib

Detailed Notes with Architecture, Commands & Practice Examples

This document covers all 10 phases of your Big Data learning roadmap with detailed theory, architecture diagrams (text-based), commands, code examples, and interview tips. Use it as your primary reference throughout your learning journey.

Table of Contents

Big Data Fundamentals

What is Big Data?

Big Data refers to datasets that are so large, fast-moving, or complex that traditional data processing tools (like relational databases or spreadsheet software) cannot handle them efficiently. The term was popularized in the early 2000s and has since become a cornerstone of modern data engineering.

Examples of Big Data sources:

- Social media posts (Facebook: 500 TB/day)
- IoT sensor data from millions of devices
- E-commerce transactions (Amazon: millions/hour)
- Server logs from web applications
- Genomics and medical imaging data
- Financial market tick data

The 5 Vs of Big Data

V	Name	Description	Example
V1	Volume	Sheer size of data — terabytes to petabytes	Facebook stores 100+ PB of photos
V2	Velocity	Speed at which data is generated and processed	Twitter: 500K tweets/minute
V3	Variety	Different formats: structured, semi, unstructured	Text, JSON, images, videos, CSV
V4	Veracity	Trustworthiness and quality of data	Social media data has noise & duplicates
V5	Value	Extracting business insights from data	Recommender systems, fraud detection

Structured vs Unstructured Data

Type	Description	Storage	Examples
Structured	Fixed schema, rows & columns	RDBMS, Hive, HBase	MySQL tables, CSV files
Semi-structured	Flexible schema, self-describing	MongoDB, Elasticsearch	JSON, XML, Avro, Parquet
Unstructured	No predefined format	HDFS, S3, NoSQL	Images, videos, emails, logs

Batch Processing vs Stream Processing

Understanding when to use each processing paradigm is critical for designing data pipelines.

Aspect	Batch Processing	Stream Processing
Definition	Process large chunks of data at scheduled intervals	Process data continuously as it arrives
Latency	High (minutes to hours)	Low (milliseconds to seconds)
Use Case	Daily reports, ETL pipelines, billing	Fraud detection, live dashboards, alerts
Tools	Hadoop MapReduce, Hive, Spark Batch	Kafka Streams, Spark Streaming, Flink
Complexity	Lower — simpler fault tolerance	Higher — stateful processing needed
Data Size	Massive historical datasets	Continuous small chunks (micro-batches)

■ **Tip:** Use Batch when accuracy matters more than speed. Use Stream when you need real-time decisions.

Distributed Systems

A distributed system is a collection of independent computers that appear to users as a single coherent system. Big Data tools are built on distributed system principles.

Key Concepts

- **CAP Theorem:** A distributed system can guarantee only 2 of 3: Consistency, Availability, Partition Tolerance
- **Fault Tolerance:** System continues operating even when nodes fail
- **Replication:** Data copied across multiple nodes to prevent data loss
- **Consistency:** All nodes see the same data at the same time
- **Partitioning:** Data split across nodes for parallel processing

Horizontal vs Vertical Scaling

Aspect	Horizontal Scaling (Scale Out)	Vertical Scaling (Scale Up)
Approach	Add more machines to the cluster	Add more CPU/RAM to existing machine
Cost	Cheaper commodity hardware	Expensive high-end servers
Limit	Theoretically unlimited	Physical hardware limits
Fault Tolerance	High — other nodes take over	Low — single point of failure
Big Data Usage	Hadoop, Spark, Kafka all use this	Traditional RDBMS approach

■ **Note:** Big Data systems like Hadoop are designed for horizontal scaling — add cheap commodity servers to grow capacity.

Fault Tolerance in Big Data

Fault tolerance ensures the system continues to work correctly even when hardware or software fails. In a 1000-node cluster, hardware failures are not exceptions — they are daily occurrences.

- Data Replication: Every block stored on 3 nodes by default (configurable)
- Heartbeat Mechanism: NameNode checks DataNode health every 3 seconds
- Re-replication: If a node dies, NameNode automatically creates new replicas
- Job Recovery: MapReduce/Spark re-executes failed tasks on healthy nodes
- Checkpointing: Save state periodically so recovery is fast

Hadoop Fundamentals

What is Apache Hadoop?

Apache Hadoop is an open-source framework for distributed storage and processing of large datasets using clusters of commodity hardware. Inspired by Google's GFS and MapReduce papers (2003-2004), Doug Cutting and Mike Cafarella created Hadoop in 2005. The name comes from Doug's son's toy elephant.

Why Hadoop Exists — The Problem It Solves

- Traditional databases couldn't handle petabyte-scale data
- Single-machine processing was too slow for massive datasets
- Expensive SAN/NAS storage was not cost-effective
- No fault-tolerant distributed file system existed for commodity hardware
- Need to process structured AND unstructured data at scale

Hadoop Ecosystem

Component	Category	Purpose
HDFS	Storage	Distributed file system — stores data across cluster
MapReduce	Processing	Batch processing framework — map then reduce
YARN	Resource Mgmt	Yet Another Resource Negotiator — cluster resource manager
Hive	SQL Layer	SQL-like queries on HDFS data
HBase	NoSQL DB	Real-time read/write on Hadoop
Pig	Scripting	High-level data flow scripting language
Sqoop	Data Transfer	Import/export between RDBMS and HDFS
Flume	Data Ingestion	Collect and move log/event data to HDFS
Oozie	Workflow	Job scheduling and workflow management
ZooKeeper	Coordination	Distributed coordination service
Spark	Fast Processing	In-memory distributed computing framework

Component	Category	Purpose
Kafka	Streaming	Distributed message broker for real-time data

Hadoop Cluster Architecture

Hadoop follows a Master-Slave (now called Primary-Worker) architecture:

HDFS Components in Detail

NameNode (Master)

- Manages the filesystem namespace — directory tree of all files
- Stores metadata: file names, permissions, block locations, replication factor
- Does NOT store actual data — only metadata
- Runs on the master server with high RAM requirements

- Single point of failure in classic Hadoop (solved by HA NameNode in Hadoop 2+)
- Block size default: 128 MB (configurable)
- Maintains: FsImage (snapshot of metadata) and EditLog (recent changes)

DataNode (Worker)

- Stores actual data blocks on local disk
- Sends heartbeat to NameNode every 3 seconds
- Sends block report to NameNode every 6 hours
- Handles read/write requests from clients directly
- Multiple DataNodes per rack for fault tolerance

Secondary NameNode

■ **Note:** Secondary NameNode is NOT a backup NameNode — this is a common misconception!

- Performs periodic checkpointing of NameNode metadata
- Merges FsImage with EditLog to prevent EditLog from growing too large
- Runs on a separate machine
- If NameNode fails, Secondary NameNode data can help recovery (but is not automatic)
- In production, use HA NameNode with ZooKeeper instead

HDFS Data Replication

Default replication factor is 3. Every block is stored on 3 different DataNodes.

```
REPLICATION STRATEGY (Default Replication = 3)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
File: bigdata.csv (300 MB)
Block size: 128 MB

Blocks created:
  Block A (0-128 MB)   → DataNode1, DataNode3, DataNode7
  Block B (128-256 MB) → DataNode2, DataNode4, DataNode8
  Block C (256-300 MB) → DataNode5, DataNode6, DataNode2

If DataNode1 dies → Block A still available on DataNode3, DataNode7
NameNode detects missing replica → creates new copy on DataNode9
```

Rack Awareness

Hadoop is aware of the network topology (racks) to optimize placement:

- Replica 1: Same node as the writer (or nearby node)
- Replica 2: Different rack from Replica 1
- Replica 3: Same rack as Replica 2 but different node

■ **Tip:** This ensures that even if an entire rack fails (power outage, switch failure), data is still available.

HDFS Commands

HDFS Command Structure

All HDFS commands follow the pattern: hdfs dfs -[command] [options] [path]

```
hdfs dfs -[command] [source] [destination]
```

Essential HDFS Commands

Command	Purpose	Example
-ls	List files/directories	hdfs dfs -ls /user/data/ hdfs dfs -ls -R / (recursive)
-mkdir	Create directory	hdfs dfs -mkdir /user/mydata hdfs dfs -mkdir -p /a/b/c (create parents)
-put / -copyFromLocal	Upload local file to HDFS	hdfs dfs -put localfile.csv /user/data/ hdfs dfs -put *.csv /user/data/
-get / -copyToLocal	Download HDFS file to local	hdfs dfs -get /user/data/file.csv ./local/ hdfs dfs -copyToLocal /user/data/ ~/downloads/
-cat	Print file contents	hdfs dfs -cat /user/data/file.txt hdfs dfs -cat /logs/2024/*.log
-rm	Delete file or directory	hdfs dfs -rm /user/data/old.csv hdfs dfs -rm -r /user/data/ (recursive) hdfs dfs -rm -r -skipTrash /tmp/
-cp	Copy within HDFS	hdfs dfs -cp /src/file.csv /dst/file.csv
-mv	Move within HDFS	hdfs dfs -mv /old/path /new/path
-chmod	Change permissions	hdfs dfs -chmod 755 /user/data/ hdfs dfs -chmod -R 777 /public/
-chown	Change ownership	hdfs dfs -chown hadoop:hadoop /user/data/
-du	Show disk usage	hdfs dfs -du -h /user/data/ hdfs dfs -du -s /user/data/ (summary)
-df	Show disk free space	hdfs dfs -df -h /
-count	Count files/dirs/bytes	hdfs dfs -count /user/data/
-stat	Print file statistics	hdfs dfs -stat '%b %n %r' /user/data/file.csv
-tail	Show last 1KB of file	hdfs dfs -tail /user/data/file.csv
-touchz	Create empty file	hdfs dfs -touchz /user/data/empty.txt
-setrep	Set replication factor	hdfs dfs -setrep -w 2 /user/data/file.csv
-text	Decode compressed files	hdfs dfs -text /user/data/file.gz

Command	Purpose	Example
-expunge	Empty trash	hdfs dfs -expunge
-checksum	Get checksum	hdfs dfs -checksum /user/data/file.csv

Admin Commands

```
# Check HDFS health
hdfs dfsadmin -report

# Check safe mode
hdfs dfsadmin -safemode get

# Leave safe mode
hdfs dfsadmin -safemode leave

# Balance data across nodes
hdfs balancer -threshold 10

# File system check
hdfs fsck / -files -blocks -locations

# NameNode format (ONLY for fresh install)
hdfs namenode -format
```

Practical Exercise: File Operations

```
# Step 1: Create sample data locally
echo "Alice,25,Engineer" > employees.csv
echo "Bob,30,Manager" >> employees.csv
echo "Carol,28,Analyst" >> employees.csv

# Step 2: Create HDFS directory
hdfs dfs -mkdir -p /user/practice/data

# Step 3: Upload file
hdfs dfs -put employees.csv /user/practice/data/

# Step 4: Verify upload
hdfs dfs -ls /user/practice/data/
hdfs dfs -cat /user/practice/data/employees.csv

# Step 5: Check file info
hdfs dfs -stat '%b %n %r %o' /user/practice/data/employees.csv
# Output: 54 employees.csv 3 134217728
# (size, name, replication, block_size)

# Step 6: Download back
hdfs dfs -get /user/practice/data/employees.csv ./downloaded.csv
```

```
# Step 7: Clean up
hdfs dfs -rm -r /user/practice/
```

MapReduce

What is MapReduce?

MapReduce is a programming model and processing framework for distributed computation of large datasets. Inspired by the Map and Reduce functions in functional programming, it was introduced by Google in 2004. It breaks a large job into smaller tasks that run in parallel across the cluster.

MapReduce Architecture

Core Components

1. Mapper

The Mapper processes each input record and emits zero or more key-value pairs.

```
# Python MapReduce (Hadoop Streaming)
# mapper.py
import sys

for line in sys.stdin:
    line = line.strip()
    words = line.split()
    for word in words:
        print(f"{word}\t1")   # key=word, value=1
```

2. Reducer

The Reducer receives sorted key-value pairs from all mappers and aggregates them.

```
# reducer.py
import sys
from itertools import groupby
from operator import itemgetter

def read_mapper_output(file, separator='\t'):
    for line in file:
        yield line.strip().split(separator, 1)

data = read_mapper_output(sys.stdin)
for current_word, group in groupby(data, itemgetter(0)):
    total_count = sum(int(count) for _, count in group)
    print(f"{current_word}\t{total_count}")
```

3. Shuffle and Sort

- Happens automatically between Map and Reduce phases
- Map outputs are partitioned by key (using hash partitioner by default)
- Each partition goes to one reducer
- Within each partition, data is sorted by key
- This ensures all values for a key arrive at the same reducer in sorted order

4. Partitioner

Determines which reducer receives each key-value pair from the mapper.

```
# Custom Partitioner in Java
public class CustomPartitioner extends Partitioner {
    @Override
    public int getPartition(Text key, IntWritable value, int numReducers
```

```
) {
        // Send keys starting with A-M to reducer 0, N-Z to reducer 1
        char firstChar = key.toString().charAt(0);
        if (firstChar <= 'M') return 0;
        else return 1 % numReducers;
    }
}
```

5. Combiner (Mini-Reducer)

A Combiner is an optional local aggregation step that runs on the map output before shuffle. It reduces network traffic significantly.

```
WITHOUT Combiner:
  Node1: (the,1),(cat,1),(the,1),(dog,1),(the,1)  → All sent to reducer
  Network transfer: 5 records

WITH Combiner:
  Node1 runs local aggregation:
  (the,1),(cat,1),(the,1),(dog,1),(the,1)
          ↓ combiner
  (cat,1),(dog,1),(the,3)   → Only 3 records sent
  Network transfer: 3 records (40% reduction)
```

■ **Tip:** Use Combiner whenever your reduce operation is commutative and associative (sum, count, max, min).

6. Input Splits

- InputSplit is a logical chunk of data assigned to one map task
- Default split size = HDFS block size (128 MB)
- More splits = more parallelism but more overhead
- InputFormat class determines how splits are created
- TextInputFormat: one record per line (default)
- SequenceFileInputFormat: binary key-value pairs
- NLineInputFormat: N lines per split

Complete Word Count Example

```
# Run via Hadoop Streaming:
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
  -input /user/data/books/ \
  -output /user/results/wordcount/ \
  -mapper mapper.py \
  -reducer reducer.py \
  -file mapper.py \
  -file reducer.py

# Check output:
```

```
hdfs dfs -cat /user/results/wordcount/part-00000 | sort -k2 -rn | head -20
```

Log Analysis Example

```
# log_mapper.py — Count HTTP status codes from Apache logs
import sys, re

for line in sys.stdin:
    # Apache log format: IP - - [date] "METHOD URL HTTP" STATUS SIZE
    match = re.search(r'" (\d{3}) ', line)
    if match:
        status_code = match.group(1)
        print(f"{status_code}\t1")

# log_reducer.py — Sum counts per status code
import sys
from collections import defaultdict

counts = defaultdict(int)
for line in sys.stdin:
    code, count = line.strip().split('\t')
    counts[code] += int(count)

for code, total in sorted(counts.items()):
    print(f"HTTP {code}: {total} requests")
```

Average Calculator Example

```
# avg_mapper.py — Emit (department, salary)
import sys, csv

reader = csv.reader(sys.stdin)
for row in reader:
    if len(row) >= 3:
        dept, salary = row[1], row[2]
        try:
            print(f"{dept}\t{float(salary)}")
        except ValueError:
            pass

# avg_reducer.py — Calculate average salary per department
import sys

current_dept = None
total = 0
count = 0
```

```
for line in sys.stdin:
    dept, salary = line.strip().split('\t')
    salary = float(salary)
    if dept == current_dept:
        total += salary
        count += 1
    else:
        if current_dept:
            print(f"{current_dept}\t{total/count:.2f}")
        current_dept = dept
        total = salary
        count = 1

if current_dept:
    print(f"{current_dept}\t{total/count:.2f}")
```

Apache Hive

What is Apache Hive?

Hive is a data warehouse infrastructure built on top of Hadoop. It provides a SQL-like language called HiveQL (HQL) to query data stored in HDFS. Facebook developed Hive in 2007 to allow analysts who knew SQL to work with Hadoop without writing Java MapReduce programs.

Hive Architecture

Managed vs External Tables

Aspect	Managed (Internal) Table	External Table
Data Ownership	Hive owns and manages the data	Data exists outside Hive control
DROP behavior	Drops table AND deletes HDFS data	Drops table metadata only, data stays
Location	Default warehouse: /user/hive/warehouse/	Custom HDFS path specified by user
Use Case	Intermediate processing tables	Shared data used by multiple tools
Best For	Temporary or Hive-only data	Production raw data, shared with Spark/Pig

```
-- Managed Table
CREATE TABLE employees (
  id INT, name STRING, dept STRING, salary DOUBLE
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

-- External Table
CREATE EXTERNAL TABLE sales_data (
  order_id INT, product STRING, amount DOUBLE, sale_date STRING
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
LOCATION '/user/shared/sales/'
STORED AS PARQUET;
```

Partitioning

Partitioning divides a table into parts based on column values, allowing Hive to skip irrelevant partitions during queries — dramatically improving performance.

```
-- Create partitioned table
CREATE TABLE logs (
  log_id INT,
  message STRING,
  log_level STRING
)
PARTITIONED BY (year INT, month INT, day INT)
STORED AS PARQUET;

-- Add data to a partition
INSERT INTO logs PARTITION (year=2024, month=1, day=15)
VALUES (1, 'System started', 'INFO');

-- Query only January 2024 — skips all other partitions
SELECT * FROM logs
WHERE year=2024 AND month=1
LIMIT 100;
```

```
-- Show partitions
SHOW PARTITIONS logs;
-- Output: year=2024/month=1/day=15
```

■ **Tip:** Without partitioning, a query on 1 day of data would scan the entire table (365x more I/O).

Bucketing

Bucketing distributes data into a fixed number of files based on a hash of a column. Unlike partitioning (splits by value), bucketing splits by hash — useful for even distribution.

```
-- Create bucketed table (32 buckets on user_id)
CREATE TABLE user_activity (
  user_id INT, event STRING, timestamp BIGINT
)
CLUSTERED BY (user_id) INTO 32 BUCKETS
STORED AS ORC;

-- Enable bucketing
SET hive.enforce.bucketing = true;

-- Bucketed JOIN (much faster — matching buckets join directly)
SELECT a.user_id, a.event, b.name
FROM user_activity a
JOIN users b ON a.user_id = b.user_id;
```

Hive Joins

Join Type	Description	Syntax
INNER JOIN	Only matching rows from both tables	FROM a JOIN b ON a.id = b.id
LEFT OUTER JOIN	All rows from left + matching from right	FROM a LEFT OUTER JOIN b ON a.id = b.id
RIGHT OUTER JOIN	All rows from right + matching from left	FROM a RIGHT OUTER JOIN b ON a.id = b.id
FULL OUTER JOIN	All rows from both tables	FROM a FULL OUTER JOIN b ON a.id = b.id
CROSS JOIN	Cartesian product of both tables	FROM a CROSS JOIN b
MAP JOIN	Small table fits in memory — no shuffle	/*+ MAPJOIN(b) */ FROM a JOIN b ON...

User Defined Functions (UDF)

```
-- Step 1: Write Python UDF
# udf_upper.py
import sys

for line in sys.stdin:
    print(line.strip().upper())

-- Step 2: Use in Hive
ADD FILE /local/path/udf_upper.py;

SELECT TRANSFORM(name)
USING 'python udf_upper.py'
AS (upper_name)
FROM employees;

-- Java UDF (production use)
-- Extend UDF class and implement evaluate()
-- Register: CREATE FUNCTION my_udf AS 'com.example.MyUDF'
--           USING JAR 'hdfs:///jars/myudf.jar';
```

Apache HBase

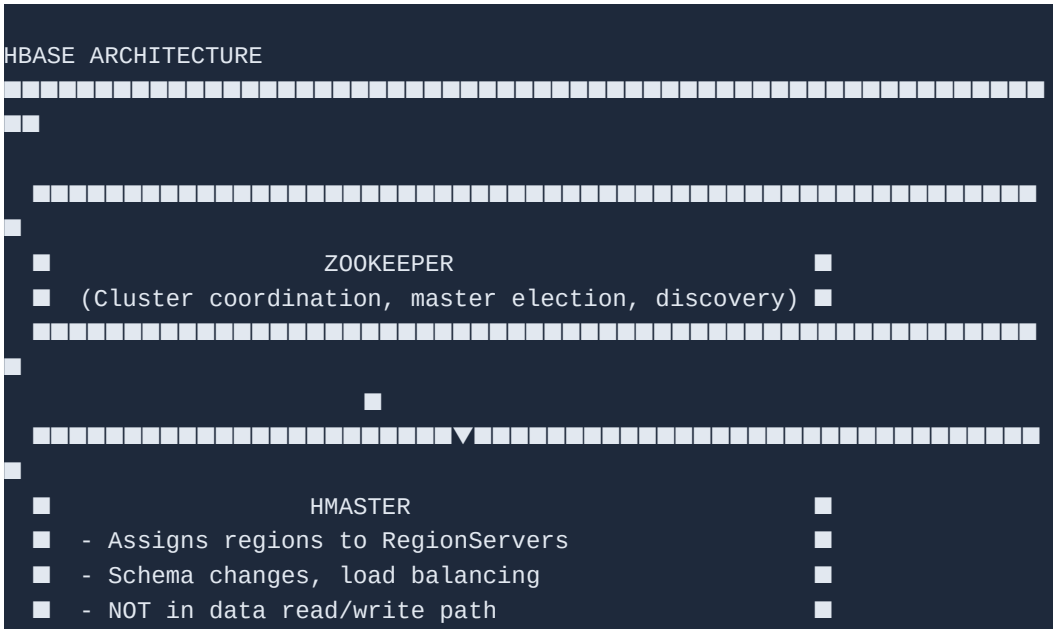
What is HBase?

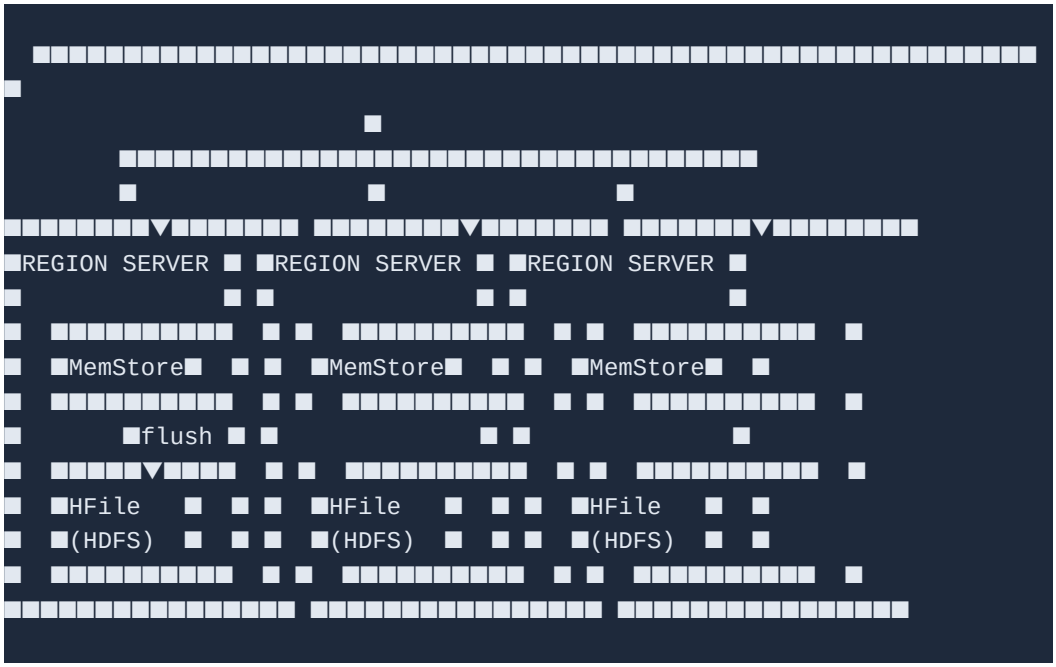
HBase is a distributed, scalable, NoSQL database modeled after Google's Bigtable. It runs on top of HDFS and provides random real-time read/write access to big data. Unlike Hive (batch), HBase supports millisecond-level lookups.

HBase Data Model

Concept	Description	Example
Table	Collection of rows	users, orders, events
Row Key	Unique identifier for each row (lexicographically sorted)	user:12345, order:20240115:001
Column Family	Group of related columns (defined at table creation)	personal:, contact:, metrics:
Column Qualifier	Column within a family (dynamic, defined at insert)	personal:name, personal:age
Cell	Intersection of row + column family + qualifier	Value + timestamp
Timestamp	Version marker — HBase stores multiple versions of a cell	1705312800000 (Unix ms)

HBase Architecture





HBase Shell Commands

```
# Start HBase shell
hbase shell

# ═══ DDL ═══════════════════════════════════════════════════
# Create table with column families
create 'users', 'personal', 'contact', 'metrics'

# List tables
list

# Describe table schema
describe 'users'

# Disable / Enable table
disable 'users'
enable 'users'

# Drop table (must disable first)
disable 'users'
drop 'users'

# ═══ DML: WRITE ════════════════════════════════════════════
# Put (insert/update): put 'table', 'rowkey', 'cf:col', 'value'
put 'users', 'user:001', 'personal:name', 'Alice'
put 'users', 'user:001', 'personal:age', '28'
put 'users', 'user:001', 'contact:email', 'alice@example.com'
put 'users', 'user:001', 'metrics:logins', '42'

# ═══ DML: READ ═════════════════════════════════════════════
# Get single row
```

```
get 'users', 'user:001'
get 'users', 'user:001', 'personal:name'

# Scan entire table
scan 'users'

# Scan with filters
scan 'users', {LIMIT => 10}
scan 'users', {STARTROW => 'user:001', ENDROW => 'user:005'}
scan 'users', {FILTER => "ValueFilter(=, 'binary:Alice')"}
scan 'users', {COLUMNS => ['personal:name', 'contact:email']}

# ■■■ DELETE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
delete 'users', 'user:001', 'personal:age'
deleteall 'users', 'user:001'

# ■■■ ADMIN ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Row count
count 'users'

# Table status
status 'detailed'

# Version history (if versions > 1 configured)
get 'users', 'user:001', {COLUMN => 'personal:age', VERSIONS => 3}
```

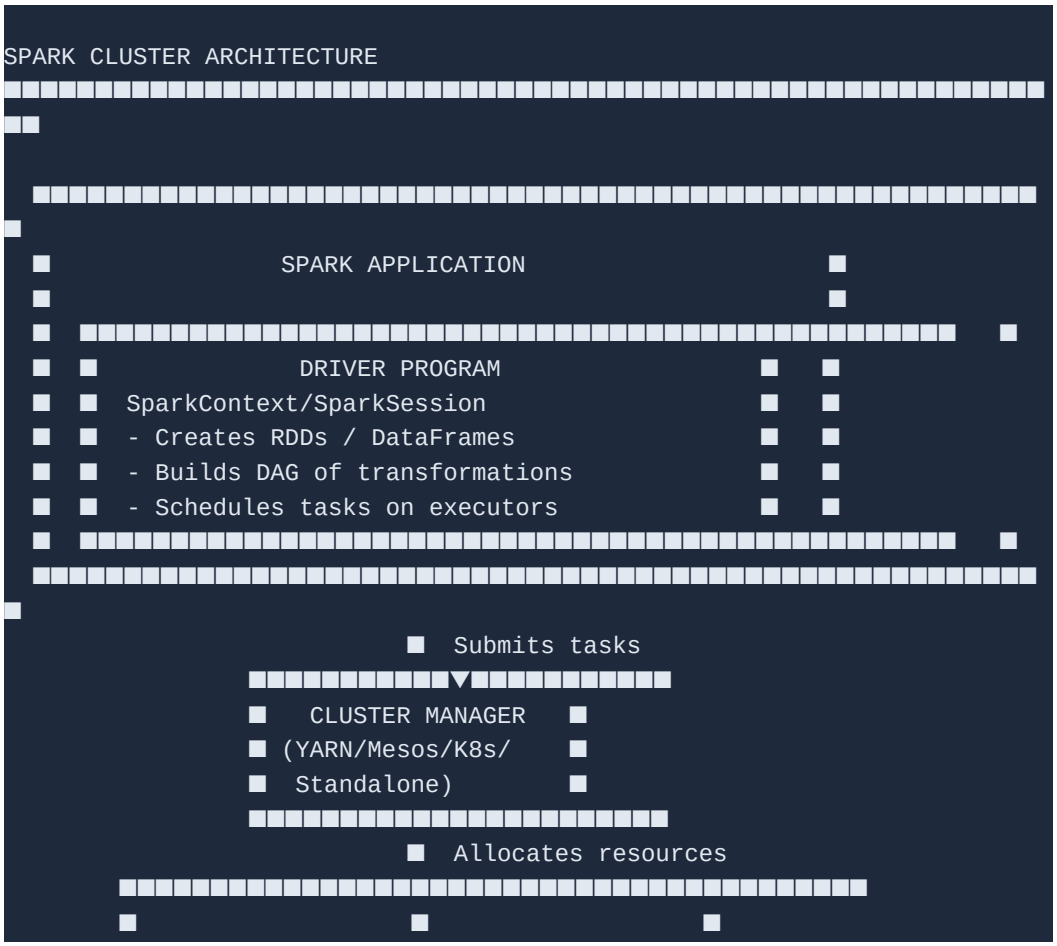
■ **Tip:** Design row keys carefully! HBase stores rows in sorted order by row key. Poor row key design leads to RegionServer hotspotting.

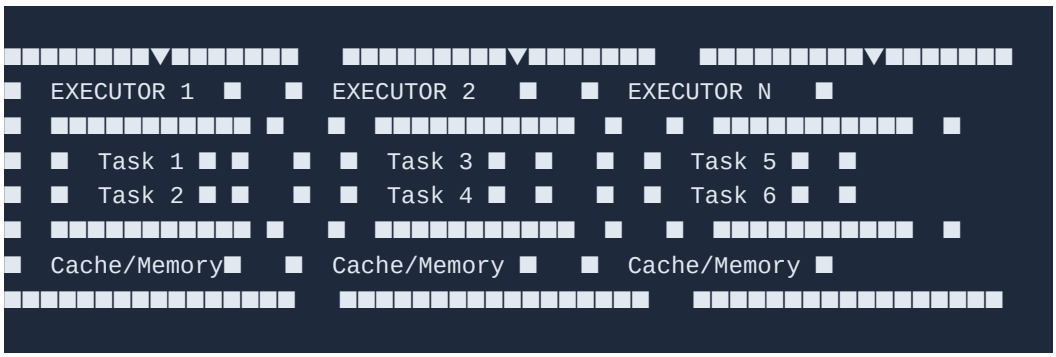
Apache Spark + PySpark

Why Spark? (vs MapReduce)

Aspect	MapReduce	Apache Spark
Speed	Slow — writes to disk after every step	100x faster — keeps data in memory (RAM)
Ease of Use	Verbose Java code	Python/Scala/R APIs available
Processing	Batch only	Batch + Stream + ML + Graph
Fault Tolerance	Re-run from HDFS	RDD lineage — recompute lost partitions
Iterative Algorithms	Terrible — re-reads disk every iteration	Excellent — cache data in memory
Language	Java primarily	Python, Scala, Java, R, SQL

Spark Architecture





RDD — Resilient Distributed Dataset

RDD is the fundamental data structure of Spark. It is an immutable, distributed collection of objects partitioned across nodes. 'Resilient' means it can recompute lost partitions using lineage.

Transformations vs Actions

Category	Meaning	Examples	When Executed
Transformation	Creates new RDD/DataFrame — lazy	map, filter, flatMap, groupBy, join	NOT immediately — builds DAG
Action	Returns result to driver or writes to storage	collect, count, take, show, write, save	Triggers actual computation

Key RDD Operations

```
from pyspark import SparkContext, SparkConf

conf = SparkConf().setAppName("RDDDemo").setMaster("local[*]")
sc = SparkContext(conf=conf)

# Create RDD
rdd = sc.parallelize([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
text_rdd = sc.textFile("hdfs:///user/data/books/*.txt")

# Transformations (lazy)
doubled = rdd.map(lambda x: x * 2)
evens = rdd.filter(lambda x: x % 2 == 0)
words = text_rdd.flatMap(lambda line: line.split(" "))
pairs = words.map(lambda w: (w, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
sorted_counts = counts.sortBy(lambda x: x[1], ascending=False)

# Actions (trigger execution)
result = sorted_counts.take(10)
total = rdd.count()
first = rdd.first()
all_data = sorted_counts.collect()   # WARNING: pulls all data to driver

# Persist / Cache
```

```
counts.cache()      # Store in memory
counts.persist()    # Default: MEMORY_ONLY

# Save
counts.saveAsTextFile("hdfs:///user/results/wordcount/")
```

DataFrames and Spark SQL

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum, avg, count, when, lit, udf
from pyspark.sql.types import *

spark = SparkSession.builder \
    .appName("DataFrameDemo") \
    .config("spark.sql.shuffle.partitions", "200") \
    .getOrCreate()

# ■■■ CREATE DATAFRAME ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
df = spark.read.csv("hdfs:///user/data/employees.csv",
                    header=True, inferSchema=True)

# From Python list
data = [("Alice", 28, "Engineering", 95000),
        ("Bob",   32, "Marketing",   72000)]
schema = StructType([
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True),
    StructField("dept", StringType(), True),
    StructField("salary", DoubleType(), True),
])
df = spark.createDataFrame(data, schema)

# ■■■ TRANSFORMATIONS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
df.show(5)
df.printSchema()
df.describe().show()

filtered = df.filter(col("salary") > 80000)
selected = df.select("name", "dept", "salary")
renamed = df.withColumnRenamed("salary", "annual_salary")
new_col = df.withColumn("bonus", col("salary") * 0.10)

# Aggregations
dept_stats = df.groupBy("dept").agg(
    count("*").alias("emp_count"),
    avg("salary").alias("avg_salary"),
    sum("salary").alias("total_salary")
)

# Sort
```

```
df.orderBy(col("salary").desc()).show()

# ■■■ JOINS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
dept_df = spark.read.csv("departments.csv", header=True)
joined = df.join(dept_df, df.dept == dept_df.dept_name, "inner")

# ■■■ SPARK SQL ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
df.createOrReplaceTempView("employees")
sql_query = (
    "SELECT dept, AVG(salary) as avg_sal, "
    "COUNT(*) as headcount "
    "FROM employees "
    "WHERE age > 25 "
    "GROUP BY dept ORDER BY avg_sal DESC"
)
result = spark.sql(sql_query)
result.show()
```

User Defined Functions (UDF)

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

# Define Python function
def salary_band(salary):
    if salary < 50000:   return "Junior"
    elif salary < 90000: return "Mid"
    else:                return "Senior"

# Register as UDF
salary_udf = udf(salary_band, StringType())

# Apply UDF
df.withColumn("band", salary_udf(col("salary"))).show()

# Pandas UDF (vectorized — much faster)
from pyspark.sql.functions import pandas_udf
import pandas as pd

@pandas_udf(StringType())
def salary_band_pandas(salary_series: pd.Series) -> pd.Series:
    return salary_series.apply(salary_band)

df.withColumn("band", salary_band_pandas(col("salary"))).show()
```

Optimization Techniques

Technique	What It Does	When to Use
Partitioning	Split data by column for parallel processing	Large datasets, join/group operations
Caching / Persist	Keep DataFrame in memory across actions	Reuse same DataFrame multiple times
Broadcast Join	Send small table to all executors — no shuffle	One table < 10 MB
Repartition	Increase partitions for more parallelism	Too few partitions, slow jobs
Coalesce	Decrease partitions (no shuffle)	Before writing small output
Predicate Pushdown	Filter early (Parquet/ORC native support)	Read only needed rows from storage
Column Pruning	Read only needed columns	Use Parquet/ORC format
AQE (Spark 3+)	Adaptive Query Execution — auto-optimizes	Always enable in Spark 3

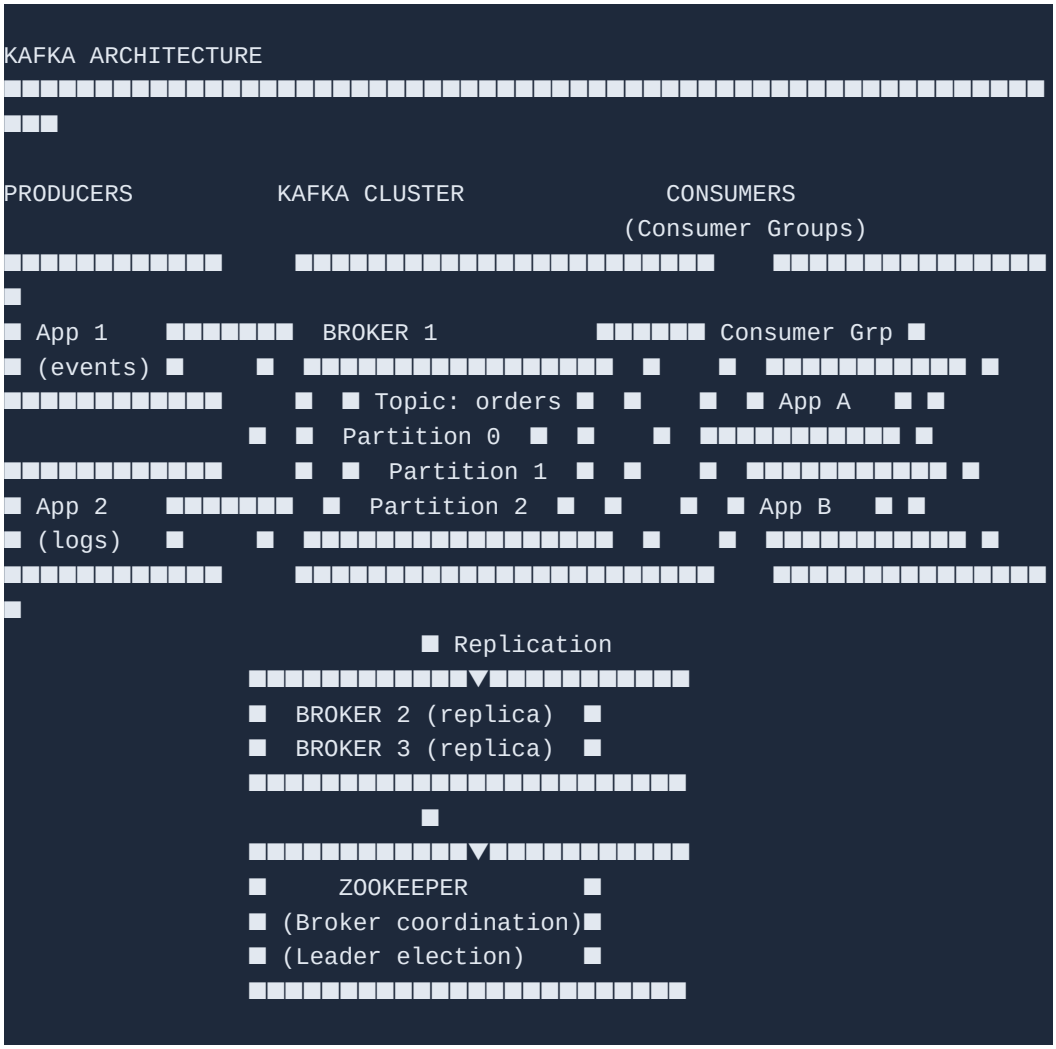
```
# Broadcast join
from pyspark.sql.functions import broadcast
result = large_df.join(broadcast(small_df), "key")

# Repartition
df_repartitioned = df.repartition(100, "dept")

# Coalesce before write
df.coalesce(1).write.csv("output/")

# Cache
df.cache()
df.count()  # Triggers caching

# Enable AQE
spark.conf.set("spark.sql.adaptive.enabled", "true")
```

Apache Kafka

What is Apache Kafka?

Apache Kafka is a distributed event streaming platform capable of handling trillions of events per day. Originally developed at LinkedIn in 2011, it is now used for real-time data pipelines, streaming analytics, event sourcing, and microservices communication.

Kafka Architecture



Core Kafka Concepts

Concept	Description
Topic	A named stream/category of messages (like a database table or queue)

Concept	Description
Partition	Topics are split into ordered, immutable sequences called partitions for parallelism
Offset	Unique sequential ID for each message within a partition — consumers track position
Producer	Application that publishes messages to topics
Consumer	Application that subscribes to topics and reads messages
Consumer Group	Group of consumers sharing a topic — each partition assigned to one consumer
Broker	A Kafka server that stores and serves messages
Leader	The broker responsible for reads/writes for a partition
Replica	Copy of a partition on another broker for fault tolerance
Retention	How long Kafka keeps messages (default 7 days, configurable by size or time)

Kafka Python Examples

```
# pip install kafka-python

# ■■■ PRODUCER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from kafka import KafkaProducer
import json, time

producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
    key_serializer=str.encode,
    acks='all',           # Wait for all replicas to acknowledge
    retries=3
)

# Send messages
for i in range(100):
    event = {
        "order_id": f"ORD-{i:04d}",
        "user_id": f"USR-{i % 10}",
        "amount": round(i * 9.99, 2),
        "timestamp": int(time.time())
    }
    producer.send("orders", key=str(i % 3), value=event)
    print(f"Sent: {event['order_id']}")

producer.flush()
producer.close()
```

```
# ■■■ CONSUMER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from kafka import KafkaConsumer

consumer = KafkaConsumer(
    'orders',
    bootstrap_servers=['localhost:9092'],
    group_id='order-processor',
    value_deserializer=lambda v: json.loads(v.decode('utf-8')),
    auto_offset_reset='earliest',   # Start from beginning
    enable_auto_commit=True
)

for message in consumer:
    order = message.value
    print(f"Partition: {message.partition}, "
          f"Offset: {message.offset}, "
          f"Order: {order['order_id']}, "
          f"Amount: ${order['amount']}")
```

Kafka CLI Commands

```
# Create topic
kafka-topics.sh --create \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --partitions 3 \
  --replication-factor 2

# List topics
kafka-topics.sh --list --bootstrap-server localhost:9092

# Describe topic
kafka-topics.sh --describe --topic orders \
  --bootstrap-server localhost:9092

# Produce messages (console)
kafka-console-producer.sh \
  --broker-list localhost:9092 \
  --topic orders

# Consume messages (console)
kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --from-beginning

# Consumer group info
kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --group order-processor \
```

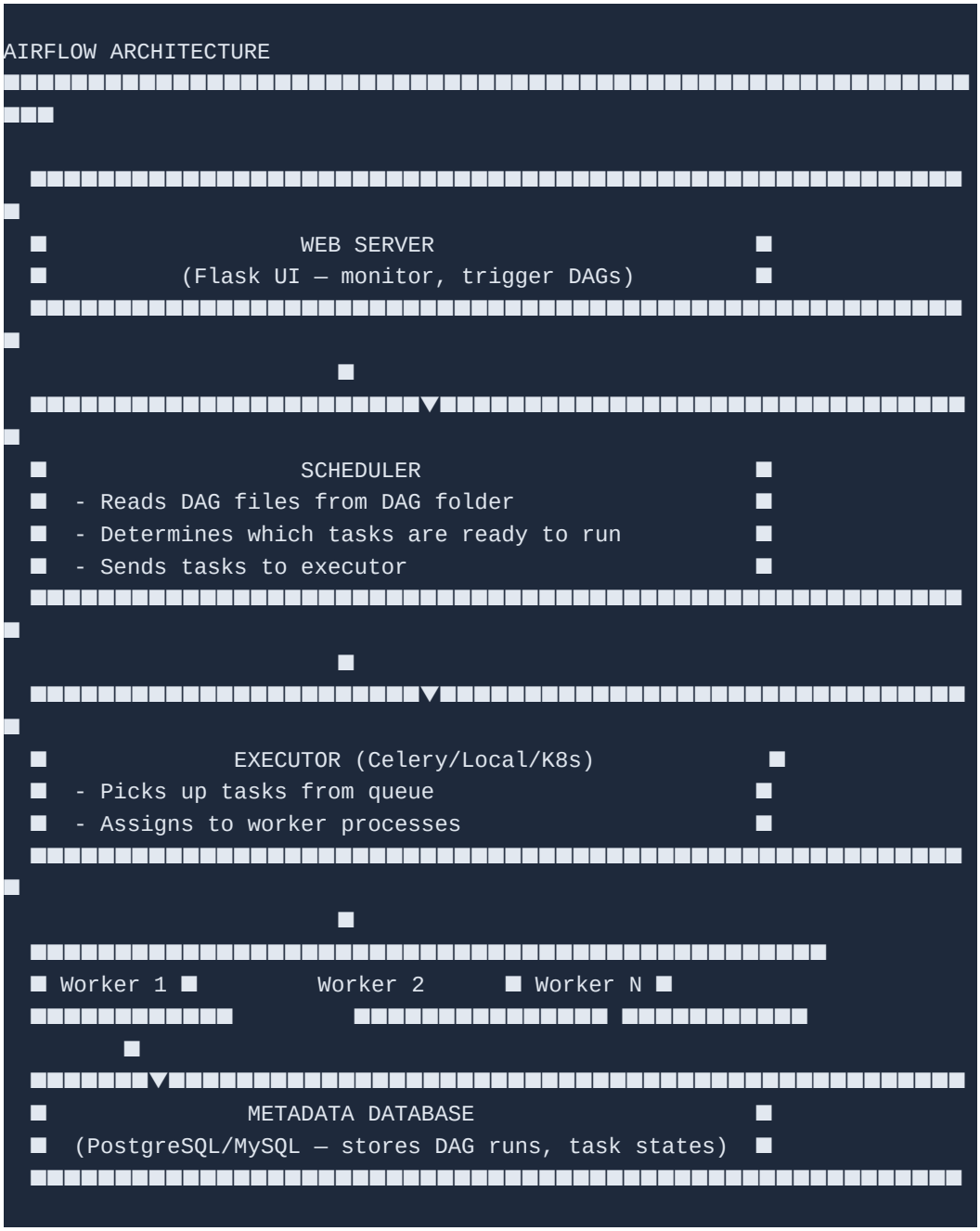
```
--describe
```

Apache Airflow

What is Apache Airflow?

Apache Airflow is an open-source platform to programmatically author, schedule, and monitor workflows. Created at Airbnb in 2014 and open-sourced in 2015, Airflow uses Python code to define workflows as Directed Acyclic Graphs (DAGs).

Airflow Architecture



DAG — Directed Acyclic Graph

A DAG is a collection of tasks organized with dependencies and relationships that describe how they should run. 'Acyclic' means no circular dependencies — tasks flow in one direction.

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator
from airflow.providers.apache.spark.operators.spark_submit import SparkS
ubmitOperator
from datetime import datetime, timedelta

# Default arguments
default_args = {
    'owner': 'data_team',
    'depends_on_past': False,
    'start_date': datetime(2024, 1, 1),
    'email_on_failure': True,
    'email': ['alerts@company.com'],
    'retries': 3,
    'retry_delay': timedelta(minutes=5),
}

# Define DAG
with DAG(
    dag_id='daily_etl_pipeline',
    default_args=default_args,
    schedule_interval='0 2 * * *',   # 2 AM daily
    catchup=False,
    tags=['etl', 'production'],
) as dag:

    # Task 1: Extract data from source
    extract = BashOperator(
        task_id='extract_from_source',
        bash_command='python /scripts/extract.py --date {{ ds }}'
    )

    # Task 2: Validate data
    def validate_data(**context):
        exec_date = context['ds']
        # Check row counts, nulls, etc.
        print(f"Validating data for {exec_date}")
        return True

    validate = PythonOperator(
        task_id='validate_data',
        python_callable=validate_data,
        provide_context=True
    )

    # Task 3: Transform with Spark
    transform = SparkSubmitOperator(
```

```
    task_id='spark_transform',
    application='/jobs/transform.py',
    conn_id='spark_default',
    application_args=['--date', '{{ ds }}']
)

# Task 4: Load to warehouse
load = BashOperator(
    task_id='load_to_warehouse',
    bash_command='python /scripts/load.py --date {{ ds }}'
)

# Task 5: Send success notification
notify = PythonOperator(
    task_id='send_notification',
    python_callable=lambda: print("Pipeline complete!")
)

# Set dependencies (execution order)
extract >> validate >> transform >> load >> notify
```

Airflow Operators Reference

Operator	Purpose	Use Case
BashOperator	Execute bash commands	Shell scripts, CLI tools
PythonOperator	Run Python functions	Custom Python logic, data validation
SparkSubmitOperator	Submit Spark jobs	Spark ETL transformations
HiveOperator	Run HiveQL queries	Hive transformations, table creation
S3ToHDFSOperator	Copy S3 files to HDFS	Cloud to Hadoop ingestion
EmailOperator	Send emails	Notifications, reports
HttpSensor	Wait for HTTP endpoint	API availability checks
FileSensor	Wait for file to appear	Wait for upstream data
BranchPythonOperator	Conditional branching	Different paths based on logic
DummyOperator	Placeholder/grouping	DAG structure organization

Spark MLlib

What is Spark MLlib?

MLlib is Spark's scalable machine learning library. It runs ML algorithms on distributed datasets using Spark's parallel processing engine, making it possible to train models on billions of records across a cluster.

MLlib Pipeline API

```
# MLlib Pipeline — chains data processing steps
from pyspark.ml import Pipeline
from pyspark.ml.feature import (
    StringIndexer, OneHotEncoder, VectorAssembler,
    StandardScaler, Imputer
)
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

# Load data
df = spark.read.csv("hdfs:///data/churn.csv", header=True, inferSchema=T
rue)
train, test = df.randomSplit([0.8, 0.2], seed=42)

# ■■ PREPROCESSING STEPS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Handle missing values
imputer = Imputer(inputCols=["age","tenure"], outputCols=["age_imp","ten
ure_imp"])

# Encode categorical column
indexer = StringIndexer(inputCol="plan", outputCol="plan_idx")
encoder = OneHotEncoder(inputCols=["plan_idx"], outputCols=["plan_vec"])

# Assemble features into a single vector
assembler = VectorAssembler(
    inputCols=["age_imp", "tenure_imp", "monthly_charges", "plan_vec"],
    outputCol="features"
)

# Scale features
scaler = StandardScaler(inputCol="features", outputCol="scaled_features"
)

# ■■ MODEL ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
rf = RandomForestClassifier(
    featuresCol="scaled_features",
```

```
    labelCol="churn",
    numTrees=100,
    maxDepth=5
)

# ■■ PIPELINE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
pipeline = Pipeline(stages=[imputer, indexer, encoder, assembler, scaler
, rf])
model = pipeline.fit(train)

# ■■ EVALUATE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
predictions = model.transform(test)
evaluator = BinaryClassificationEvaluator(labelCol="churn")
auc = evaluator.evaluate(predictions)
print(f"AUC-ROC: {auc:.4f}")

# ■■ HYPERPARAMETER TUNING ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
param_grid = ParamGridBuilder() \
    .addGrid(rf.numTrees, [50, 100]) \
    .addGrid(rf.maxDepth, [3, 5, 7]) \
    .build()

cv = CrossValidator(estimator=pipeline, estimatorParamMaps=param_grid,
                    evaluator=evaluator, numFolds=3)
cv_model = cv.fit(train)
```

Feature Engineering

Transformer	Purpose	Example
StringIndexer	Encode string labels to indices	Male/Female → 0/1
OneHotEncoder	Convert index to binary vector	0 → [1,0,0], 1 → [0,1,0]
VectorAssembler	Combine multiple columns into feature vector	[age, salary, dept_vec] → features
StandardScaler	Normalize features (mean=0, std=1)	Prevents large-scale features dominating
MinMaxScaler	Scale to [0,1] range	Useful for neural networks
PCA	Dimensionality reduction	Reduce 1000 features to 50
Imputer	Fill missing values with mean/median	Replace nulls in age column
Tokenizer	Split text into words	NLP preprocessing
TF-IDF	Text feature extraction	Document classification
Bucketizer	Convert continuous to categorical	Age → Young/Mid/Senior

ML Algorithms in MLlib

Category	Algorithm	Spark Class
Classification	Logistic Regression	LogisticRegression
Classification	Random Forest	RandomForestClassifier
Classification	Gradient Boosted Trees	GBTClassifier
Classification	Decision Tree	DecisionTreeClassifier
Regression	Linear Regression	LinearRegression
Regression	Random Forest Regression	RandomForestRegressor
Clustering	K-Means	KMeans
Recommendation	ALS (Collaborative Filtering)	ALS
Dimensionality	PCA	PCA
Text Mining	LDA (Topic Modeling)	LDA

Model Evaluation Metrics

Problem Type	Metric	Evaluator Class
Binary Classification	AUC-ROC, Accuracy, F1	BinaryClassificationEvaluator
Multi-class Classification	Accuracy, F1, Precision, Recall	MulticlassClassificationEvaluator
Regression	RMSE, MAE, R^2	RegressionEvaluator
Clustering	Silhouette Score	ClusteringEvaluator

Quick Reference & Timeline

Realistic Study Timeline

Phase	Topic	Duration	Priority
6	Big Data Fundamentals	1 week	■■■ Essential
7	Hadoop Fundamentals	3 weeks	■■■ Essential
8	HDFS Commands	1 week	■■■ Essential
9	MapReduce	3 weeks	■■■ Essential
10	Hive	2-3 weeks	■■■ Essential
11	HBase	2 weeks	■■ Important
12	Spark + PySpark	4 weeks	■■■ Critical for Jobs
13	Kafka	2 weeks	■■■ Critical for Jobs
14	Airflow	2 weeks	■■ Important
15	Spark MLlib	2-3 weeks	■■ Good to Have
	TOTAL	~25 weeks	~6 months

Top Interview Topics by Frequency

- HDFS architecture — NameNode, DataNode, replication, block size
- MapReduce flow — map, shuffle, sort, reduce, partitioner, combiner
- Hive: managed vs external table, partitioning vs bucketing
- Spark: RDD vs DataFrame vs Dataset, transformations vs actions
- Spark: why it is faster than MapReduce (in-memory, DAG optimization)
- Kafka: producer-consumer model, topic, partition, offset, consumer group
- HBase: row key design, column family, difference from RDBMS
- Difference between batch and streaming processing
- CAP theorem in distributed systems
- Spark optimization: broadcast join, caching, partitioning, AQE

Recommended Learning Resources

Resource	Type	Best For
Hadoop: The Definitive Guide (Tom White)	Book	Deep Hadoop, HDFS, MapReduce

Resource	Type	Best For
Learning Spark (O'Reilly)	Book	Comprehensive Spark coverage
Programming Hive (O'Reilly)	Book	Hive in depth
Databricks Community Edition	Practice	Free Spark cluster in cloud
Apache Hadoop docs	Official Docs	HDFS, MapReduce commands
Apache Kafka docs	Official Docs	Kafka architecture, APIs
Apache Airflow docs	Official Docs	DAG syntax, operators
Kaggle datasets	Practice Data	Real datasets for practice

Best of luck on your Big Data Engineering journey! Focus on Spark and Kafka for job readiness. Practice on Databricks Community Edition — it is free and runs in the cloud without local setup.